

DL-601

Deep Learning

Foundations, Computer Vision and Production Systems

Course Module Document

Graduate (Master's / first-year PhD)

3 credit hours - 16 lectures x 3 hours (2h lecture + 1h lab)

Instructor: Senior AI Engineer, Course Instructor

16 lectures · 16 slide decks · 16 hands-on notebooks · 126 graded tasks

Every notebook runs end to end on a CPU laptop. No downloads, no GPU, no API keys.

1. Course overview

This module takes a student with no deep learning background to the point of building, diagnosing and deploying modern neural networks. The emphasis is on images, with text introduced where it is needed to motivate sequence models and attention, and a deployment thread running through the whole course.

Curriculum design follows the consensus structure of Stanford CS231n, NYU DS-GA 1008 (LeCun and Canziani) and the deeplearning.ai Deep Learning Specialization, re-sequenced for a 16-session semester and extended with the production engineering those courses leave out.

What makes this module different: every technique is implemented from first principles before the library version is used. Students write backpropagation, convolution, attention, NMS and a diffusion sampler themselves. They then serve a model behind a tested HTTP API, because a model nobody can call is not a deliverable.

Learning outcomes

On completing this module, a student can:

1. Explain how a deep network computes its output and how gradients flow back through it.
2. Implement backpropagation, convolution and attention from first principles in NumPy and PyTorch.
3. Select and train an appropriate architecture for image classification, detection and segmentation.
4. Diagnose an underperforming model from its learning curves, confusion matrix and saliency maps.
5. Apply transfer learning and self-supervised pre-training when labelled data is scarce.
6. Build and reason about Transformers for both text and image inputs.
7. Serve a trained model behind a documented, tested FastAPI endpoint.
8. Evaluate a model for robustness, calibration and fairness before release.

Prerequisites

- Linear algebra: matrices, matrix products, eigenvalues
- Multivariable calculus: partial derivatives and the chain rule
- Probability: random variables, expectation, maximum likelihood
- Python: comfortable with functions, classes, NumPy arrays
- No prior deep learning experience assumed

Technology stack

- Python 3.10+
- PyTorch 2.x (CPU is sufficient for every notebook in this course)
- NumPy, Matplotlib, scikit-learn, Pillow, pandas
- FastAPI + Uvicorn for model serving
- Jupyter / JupyterLab

Structure

Part	Lectures	Focus
Part I - Foundations	1–4	Tensors, linear models, backpropagation, optimisation. Everything built by hand.
Part II - Convolutional Vision	5–8	Convolution, ResNets, transfer learning, augmentation, evaluation and interpretability.
Part III - Vision Systems and Deployment	9–12	Deployment with FastAPI, then detection, segmentation and sequence models.
Part IV - Attention, Generation and Production	13–16	Attention, Vision Transformers, self-supervision, generative models, production systems.

2. Lecture schedule

Part I - Foundations

- 01 **Deep Learning Foundations and the Image Data Pipeline**
Why depth works, and how pixels become tensors.
- 02 **From Linear Models to Neural Networks**
The softmax classifier, and the exact point where linearity fails.
- 03 **Backpropagation and Automatic Differentiation**
The chain rule, organised as a graph - and built from scratch.
- 04 **Training Deep Networks: Optimisation and Regularisation**
Everything between 'it runs' and 'it works'.

Part II - Convolutional Vision

- 05 **Convolutional Neural Networks**
The right inductive bias for images, derived rather than asserted.
- 06 **Modern CNN Architectures and Transfer Learning**
ResNet, and why fine-tuning beats training from scratch almost every time.
- 07 **Data Augmentation, Training Recipes and Experiment Tracking**
The unglamorous work that produces most of the accuracy.
- 08 **Evaluation, Interpretability and Model Debugging**
Accuracy is one number. It is rarely the one you need.

Part III - Vision Systems and Deployment

- 09 **Serving Deep Learning Models with FastAPI**
A model that nobody can call is a model that does not exist.
- 10 **Object Detection**
From 'what is in this image' to 'what, and exactly where'.
- 11 **Semantic Segmentation and Dense Prediction**
One prediction per pixel, and the encoder-decoder that makes it possible.
- 12 **Sequence Models: RNNs, LSTMs and Image Captioning**
Memory, and the reason it eventually was not enough.

Part IV - Attention, Generation and Production

13 Attention and the Transformer

Scaled dot-product attention, built from nothing.

14 Vision Transformers and Self-Supervised Learning

Images as sequences of patches, and learning without labels.

15 Generative Models: Autoencoders, VAEs, GANs and Diffusion

Learning the distribution, not just the decision boundary.

16 Multimodal Learning and Production ML Systems

Joining vision to language, and shipping the result responsibly.

3. Assessment and policies

Assessment breakdown

Component	Weight	Description
Weekly lab tasks (16 x 1.5%)	24%	Graded on completion and correctness of the core tasks.
Assignment 1 - Foundations (Lectures 1-4)	12%	Backpropagation from scratch and an MLP training study.
Assignment 2 - Convolutional Vision (Lectures 5-8)	14%	CNN design, transfer learning and error analysis.
Assignment 3 - Sequences and Attention (Lectures 10-14)	14%	Detection or segmentation plus a Transformer built from scratch.
Capstone project (Lectures 15-16)	26%	End-to-end system: data, training, evaluation, FastAPI service, report.
Participation and paper discussion	10%	One 10-minute paper presentation per student.

Per-task marking rubric

Criterion	Weight	Full marks
Correctness	40%	Every core task produces the required output, and the numbers are right.
Code quality	20%	Readable, functions documented, no copy-pasted blocks, no dead code.
Analysis	25%	Written answers explain why, not just what. Claims backed by a number or a plot.
Reproducibility	15%	Runs top to bottom from a clean kernel with the seed fixed.

Grading scale

A	A-	B+	B	B-	C+	C	F
93-100	90-92	87-89	83-86	80-82	77-79	70-76	below 70

Policies

- Late work: 10% per day, up to three days; after that the task scores zero.
- Collaboration: discussion is encouraged, submitted code and text must be your own.
- AI assistants: permitted for explanation and debugging, must be disclosed in a short note; generating an entire solution is not permitted.
- Reproducibility: every submission must run end to end from a clean checkout with a fixed seed.
- Compute: no GPU is required. If a notebook takes more than ~10 minutes on your laptop, reduce the epoch count and say so in your write-up.

Weekly rhythm

Block	Duration	Activity
Lecture	2 h	Slide deck, worked derivations, live code from the notebook.
Lab	1 h	Supervised work on the notebook; instructor circulates.
Independent	4–6 h	Core tasks, reading, and the running results table.

4. Repository layout

```

DeepLearning/
├─ Deep_Learning_Course_Module.pdf    this document
├─ README.md                         start here
├─ requirements.txt
├─ dlcourse/                          shared helpers imported by every notebook
│   ├── data.py                      dataset loaders
│   ├── training.py                  the reusable training loop
│   └─ viz.py                        plotting helpers
├─ data/                              generated offline; no downloads, ever
│   ├── shapes_32.npz                4-class 32x32 classification set
│   ├── shapes_imagefolder/          the same data as PNG files
│   ├── digits_8x8.npz               scikit-learn handwritten digits
│   ├── shapes_det.npz + annotations multi-object detection scenes
│   ├── shapes_seg.npz               per-pixel segmentation masks
│   ├── shapes_pairs.npz             unlabelled pool for self-supervision
│   ├── sentiment.csv, corpus.txt    text corpora
│   └─ captions.csv                 image-caption pairs
├─ lectures/
│   └─ Lecture_01 ... Lecture_16/
│       ├── README.md                objectives, outline, teaching notes
│       ├── slides/*.pptx            the lecture deck
│       ├── notebooks/*.ipynb        the hands-on lab
│       ├── tasks/*_Tasks.md         student tasks and rubric
│       ├── data/README.md           which datasets this lecture uses
│       └─ app/                      FastAPI service (Lectures 9 and 16)
└─ tools/                            generators for every artefact above

```

Getting started

```

# 1. install dependencies
pip install -r requirements.txt

# 2. generate the datasets (once, ~30 seconds, fully offline)
python tools/build_data.py

# 3. open the first lab
jupyter lab lectures/Lecture_01/notebooks/

# 4. for the serving lectures
cd lectures/Lecture_09/app
python train_and_export.py
uvicorn main:app --reload      # then open http://127.0.0.1:8000/docs

```

Design constraint: every dataset is generated procedurally by a seeded script, so the repository is self-contained and reproducible byte for byte. Nothing downloads. No notebook needs a GPU, an API key, or an internet connection.

5. Lecture specifications

Part I - Foundations

LECTURE 01 · Part I - Foundations

Deep Learning Foundations and the Image Data Pipeline

Why depth works, and how pixels become tensors.

LEARNING OBJECTIVES

- Place deep learning relative to classical machine learning and explain what changed after 2012.
- Describe an image as a tensor and move fluently between NumPy, PIL and PyTorch layouts.
- Build a Dataset and DataLoader and reason about batching, shuffling and normalisation.
- Establish a baseline and a train/validation/test protocol before touching a model.

LECTURE OUTLINE

1. **What deep learning actually is** — Representation learning: features are learned, not hand-designed.
2. **When to use deep learning - and when not to** — Good fit: perceptual data (images, audio, text), abundant data, tolerance for opacity.
3. **Images as tensors** — A colour image is a rank-3 tensor: (height, width, channels).
4. **The data pipeline** — Dataset defines `__len__` and `__getitem__`; DataLoader adds batching, shuffling and parallel workers.
5. **Splits, baselines and leakage** — Train fits parameters, validation selects hyper-parameters, test is touched exactly once.

HANDS-ON LAB

Load the shapes dataset three ways (npz arrays, a Dataset class, an on-disk image folder), visualise it, compute normalisation statistics correctly, and fit a majority-class and a linear baseline to fix the numbers the rest of the course must beat.

CORE TASKS (4 REQUIRED, 2 STRETCH)

1. Tensor layout drill
2. Custom Dataset
3. Honest normalisation
4. Baselines that matter

READING

- Goodfellow, Bengio & Courville, *Deep Learning*, Chapter 1 and Section 5.1-5.3.
- Krizhevsky, Sutskever & Hinton (2012), *ImageNet Classification with Deep CNNs* (AlexNet).
- PyTorch docs: Datasets and DataLoaders tutorial.

data: shapes_32.npz, shapes_imagefolder/

slides: 5 sections

LECTURE 02 · Part I - Foundations

From Linear Models to Neural Networks*The softmax classifier, and the exact point where linearity fails.***LEARNING OBJECTIVES**

- Derive the softmax classifier and the cross-entropy loss from maximum likelihood.
- Implement the forward pass and analytic gradient of a linear classifier in NumPy.
- Explain why stacking linear layers without a non-linearity gains nothing.
- Show empirically that a hidden layer plus ReLU solves a problem a linear model cannot.

LECTURE OUTLINE

1. **The linear score function** — $s = Wx + b$ maps a flattened image to one score per class.
2. **Softmax and cross-entropy** — Softmax turns scores into a probability distribution: $p_k = \exp(s_k) / \sum_j \exp(s_j)$.
3. **Why a non-linearity is required** — $W_2(W_1 x) = (W_2 W_1) x$ - composing linear maps yields another linear map.
4. **The universal approximation theorem, honestly** — One hidden layer of sufficient width can approximate any continuous function on a compact set.
5. **Loss surfaces and what optimisation faces** — Convex for a linear model with cross-entropy; non-convex the moment a hidden layer appears.

HANDS-ON LAB

Implement a softmax classifier in pure NumPy with an analytic gradient, verify it against a numerical gradient, train it on the digits data, then demonstrate the linear/non-linear gap on a two-moons problem and on the shapes dataset.

CORE TASKS (5 REQUIRED, 2 STRETCH)

1. Numerically stable softmax
2. Analytic gradient plus gradient check
3. Train the linear classifier
4. Visualise the templates
5. One hidden layer changes everything

READING

- Goodfellow et al., *Deep Learning*, Chapter 6.
- CS231n course notes: Linear Classification and Optimization.
- Nair & Hinton (2010), *Rectified Linear Units Improve Restricted Boltzmann Machines*.

data: digits_8x8.npz, shapes_32.npz

slides: 5 sections

LECTURE 03 · Part I - Foundations

Backpropagation and Automatic Differentiation*The chain rule, organised as a graph - and built from scratch.***LEARNING OBJECTIVES**

- Derive backpropagation as the reverse-mode application of the chain rule on a computational graph.
- Implement a working reverse-mode autodiff engine in under 150 lines of NumPy.
- Compute local gradients for the layers used throughout the course.
- Recognise vanishing and exploding gradients from the gradient-norm profile across layers.

LECTURE OUTLINE

1. **The computational graph** — Every model is a DAG of primitive operations; each node knows its local derivative.
2. **Forward mode versus reverse mode** — Forward mode costs one pass per input; reverse mode costs one pass per output.
3. **Local gradients you should know by heart** — Add: distributes the incoming gradient unchanged to both inputs.
4. **Broadcasting and the sum rule** — Where a tensor was broadcast in the forward pass, sum the gradient over the broadcast axes.
5. **When gradients go wrong** — Vanishing: repeated multiplication by factors below one drives early-layer gradients to zero.

HANDS-ON LAB

Build a Tensor class with reverse-mode autodiff from scratch (add, mul, matmul, relu, softmax cross-entropy), validate every gradient against PyTorch, train an MLP with it, then instrument a deep network to observe vanishing gradients directly.

CORE TASKS (5 REQUIRED, 2 STRETCH)

1. Hand-derive a graph
2. Finish the autodiff engine
3. Verify against PyTorch
4. Train with your own engine
5. Gradient-norm profile

READING

- Goodfellow et al., *Deep Learning*, Section 6.5.
- Baydin et al. (2018), *Automatic Differentiation in Machine Learning: a Survey*.
- Karpathy, *micrograd* - a 150-line autodiff engine worth reading line by line.

data: digits_8x8.npz, shapes_32.npz

slides: 5 sections

LECTURE 04 · Part I - Foundations

Training Deep Networks: Optimisation and Regularisation*Everything between 'it runs' and 'it works'.***LEARNING OBJECTIVES**

- Compare SGD, momentum, RMSProp and Adam, and state when each is the right default.
- Choose an initialisation scheme that keeps activation variance stable with depth.
- Apply batch normalisation, dropout and weight decay, and explain what each actually does.
- Diagnose underfitting and overfitting from learning curves and respond correctly.

LECTURE OUTLINE

1. **From gradient descent to Adam** — SGD: cheap, noisy, and the noise itself is a useful regulariser.
2. **Learning rate: the hyper-parameter that matters most** — Too high diverges; too low crawls or stalls in a bad region.
3. **Initialisation** — All zeros breaks symmetry permanently - every unit computes the same thing forever.
4. **Normalisation layers** — BatchNorm standardises each channel over the batch, then applies a learned scale and shift.
5. **Regularisation** — L2 / weight decay pulls weights toward zero and prefers smoother functions.
6. **Reading the learning curves** — High train loss and high val loss: underfitting - more capacity, longer training, higher LR.

HANDS-ON LAB

Run controlled experiments: an optimiser bake-off on identical seeds, a learning-rate range test, an initialisation comparison measured by activation variance per layer, and an ablation of BatchNorm, dropout and weight decay on a deliberately overfitting setup.

CORE TASKS (5 REQUIRED, 2 STRETCH)

1. Optimiser bake-off
2. Learning-rate range test
3. Initialisation and activation variance
4. Regularisation ablation
5. BatchNorm in train versus eval

READING

- Kingma & Ba (2015), *Adam: A Method for Stochastic Optimization*.
- He et al. (2015), *Delving Deep into Rectifiers* (He initialisation).
- Ioffe & Szegedy (2015), *Batch Normalization*.
- Loshchilov & Hutter (2019), *Decoupled Weight Decay Regularization* (AdamW).

data: shapes_32.npz

slides: 6 sections

Part II - Convolutional Vision

LECTURE 05 · Part II - Convolutional Vision

Convolutional Neural Networks*The right inductive bias for images, derived rather than asserted.***LEARNING OBJECTIVES**

- Explain convolution as local connectivity plus weight sharing, and count the parameters it saves.
- Compute output shapes for any kernel, stride, padding and dilation without guessing.
- Implement 2D convolution and max pooling from scratch, then match PyTorch's result.
- Reason about receptive field growth and design a CNN whose receptive field covers the input.

LECTURE OUTLINE

1. **Why not just use an MLP** — A 224x224x3 image into one 1000-unit hidden layer is 150 million weights in a single layer.
2. **The two ideas** — Local connectivity: a unit sees a small patch, because pixels far apart are weakly related.
3. **Convolution arithmetic** — $\text{out} = \text{floor}((\text{in} + 2 * \text{padding} - \text{dilation} * (\text{kernel} - 1) - 1) / \text{stride}) + 1$.
4. **Pooling and downsampling** — Max pooling gives small-translation invariance and reduces computation.
5. **Receptive field** — Stacking two 3x3 layers gives a 5x5 receptive field with fewer parameters than one 5x5 layer.
6. **What the filters learn** — Layer 1: oriented edges and colour blobs - remarkably close to Gabor filters, every time.

HANDS-ON LAB

Implement conv2d with im2col and max pooling from scratch and match torch.nn.functional to 1e-5; apply hand-built edge kernels to see what convolution does; train a CNN to ~99% and compare it against an MLP on accuracy, parameter count and — the decisive test — accuracy on translated images; then visualise the learned first-layer filters and measure the receptive field.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. Shape arithmetic without running code
2. Convolution from scratch
3. Max pooling forward and backward
4. Train a CNN on shapes
5. Parameter accounting
6. Visualise layer-1 filters

READING

- LeCun et al. (1998), *Gradient-Based Learning Applied to Document Recognition* (LeNet-5).
- Dumoulin & Visin (2016), *A Guide to Convolution Arithmetic for Deep Learning*.
- CS231n notes: Convolutional Neural Networks.

data: shapes_32.npz

slides: 6 sections

LECTURE 06 · Part II - Convolutional Vision

Modern CNN Architectures and Transfer Learning*ResNet, and why fine-tuning beats training from scratch almost every time.***LEARNING OBJECTIVES**

- Trace the architectural lineage from LeNet through AlexNet, VGG and Inception to ResNet.
- Explain the degradation problem and how residual connections resolve it.
- Implement a residual block and a small ResNet from scratch.
- Choose correctly between feature extraction and fine-tuning given dataset size and domain distance.

LECTURE OUTLINE

1. **The architecture lineage** — LeNet-5 (1998): the template - conv, pool, conv, pool, dense.
2. **The degradation problem** — A 56-layer plain network had higher *training* error than a 20-layer one. Not overfitting - optimisation.
3. **Building blocks worth knowing** — 1x1 convolution: channel mixing and dimensionality reduction at negligible spatial cost.
4. **Transfer learning** — Early layers learn generic features (edges, textures) that transfer across almost any image domain.
5. **Choosing a strategy** — Small data, similar domain: freeze the backbone, train a linear head.

HANDS-ON LAB

Implement a residual block and a small ResNet, empirically reproduce the degradation problem with a 20- versus 40-layer plain network, then run the transfer-learning comparison: a source model pre-trained on one shape subset, transferred to a 100-example target task, versus training from scratch.

CORE TASKS (5 REQUIRED, 2 STRETCH)

1. Residual block from scratch
2. Reproduce the degradation problem
3. Small ResNet on shapes
4. Feature extraction versus fine-tuning
5. How many layers to unfreeze

READING

- He et al. (2016), *Deep Residual Learning for Image Recognition*.
- Simonyan & Zisserman (2015), *Very Deep Convolutional Networks* (VGG).
- Yosinski et al. (2014), *How Transferable Are Features in Deep Neural Networks?*
- Howard et al. (2017), *MobileNets*.

data: shapes_32.npz, shapes_imagefolder/

slides: 5 sections

LECTURE 07 · Part II - Convolutional Vision

Data Augmentation, Training Recipes and Experiment Tracking*The unglamorous work that produces most of the accuracy.***LEARNING OBJECTIVES**

- Design an augmentation policy whose transformations preserve the label.
- Implement mixup and cutmix and explain what they regularise.
- Run a reproducible experiment: fixed seeds, logged configuration, recorded artefacts.
- Choose between grid, random and successive-halving hyper-parameter search.

LECTURE OUTLINE

1. **Augmentation as a prior** — Augmentation encodes the invariances you know the task has.
2. **The standard geometric and photometric toolkit** — Random crop with padding, horizontal flip, small rotation, scale and translate.
3. **Mixup and cutmix** — Mixup: $x = \text{lam} * x_i + (1 - \text{lam}) * x_j$ with the labels mixed identically, $\text{lam} \sim \text{Beta}(a, a)$.
4. **Test-time augmentation** — Average predictions over several augmented copies of the test image.
5. **Reproducibility in practice** — Seed Python, NumPy and PyTorch; set deterministic algorithms when you need bitwise repeatability.
6. **Hyper-parameter search** — Grid search wastes evaluations on parameters that do not matter.

HANDS-ON LAB

Build an augmentation pipeline from tensor operations, measure each transformation's effect in a controlled ablation, implement mixup and cutmix with soft-label loss, then build a small experiment tracker (JSON log + results table) and run a random search over the training recipe.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. Augmentation from scratch
2. Label-preserving audit
3. Ablation study
4. Mixup
5. Experiment tracker
6. Random search

READING

- Zhang et al. (2018), *mixup: Beyond Empirical Risk Minimization*.
- Yun et al. (2019), *CutMix*.
- Bergstra & Bengio (2012), *Random Search for Hyper-Parameter Optimization*.
- Shorten & Khoshgoftaar (2019), *A Survey on Image Data Augmentation*.

data: shapes_32.npz

slides: 6 sections

LECTURE 08 · Part II - Convolutional Vision

Evaluation, Interpretability and Model Debugging*Accuracy is one number. It is rarely the one you need.***LEARNING OBJECTIVES**

- Select metrics appropriate to class imbalance and to the cost structure of the errors.
- Assess calibration and correct it with temperature scaling.
- Implement saliency maps and Grad-CAM and read them critically.
- Follow a systematic debugging protocol instead of changing hyper-parameters at random.

LECTURE OUTLINE

1. **Beyond accuracy** — With 99% negatives, a model predicting 'negative' always scores 99%. Accuracy is worthless there.
2. **The confusion matrix is the first diagnostic** — It converts one number into a map of which classes are being confused for which.
3. **Calibration** — A well-calibrated model that says 0.8 is right 80% of the time. Modern networks are badly over-confident.
4. **Interpretability methods** — Vanilla saliency: the gradient of the class score with respect to the input pixels. Noisy but instant.
5. **Shortcut learning** — Models exploit whatever correlates with the label, including artefacts you never intended.
6. **A debugging protocol** — 1. Overfit a single batch. If the loss will not reach zero, the bug is in the model or the loss.

HANDS-ON LAB

Build a full evaluation report for the Lecture 6 model: per-class metrics, confusion matrix, highest-confidence errors, reliability diagram with temperature scaling, saliency and Grad-CAM. Then diagnose a deliberately shortcut-corrupted dataset and prove the shortcut with Grad-CAM.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. Full metric report
2. Error gallery
3. Reliability diagram and ECE
4. Temperature scaling
5. Grad-CAM
6. Catch the shortcut

READING

- Guo et al. (2017), *On Calibration of Modern Neural Networks*.
- Selvaraju et al. (2017), *Grad-CAM*.
- Adebayo et al. (2018), *Sanity Checks for Saliency Maps*.
- Geirhos et al. (2020), *Shortcut Learning in Deep Neural Networks*.

data: shapes_32.npz, shapes_det.npz

slides: 6 sections

Part III - Vision Systems and Deployment

LECTURE 09 · Part III - Vision Systems and Deployment

Serving Deep Learning Models with FastAPI

A model that nobody can call is a model that does not exist.

LEARNING OBJECTIVES

- Export a trained model and load it for inference with correct, reproducible preprocessing.
- Build a FastAPI service that accepts an image upload and returns calibrated class probabilities.
- Validate inputs with Pydantic and return useful, correct HTTP error codes.
- Measure latency and throughput, and improve both with batching and threading controls.

LECTURE OUTLINE

1. **Training code is not serving code** — Training optimises throughput on batches; serving optimises latency on single requests.
2. **Model export formats** — `state_dict`: the recommended PyTorch format, but it requires the class definition to load.
3. **FastAPI essentials** — Path operations, type hints and automatic OpenAPI documentation at `/docs`.
4. **Correctness at the edges** — Reject non-image uploads with 415, oversized files with 413, malformed input with 422.
5. **Performance** — Measure p50, p95 and p99 latency, not the mean. Tail latency is what users feel.
6. **Testing a model service** — `TestClient` gives you fast endpoint tests with no network.

HANDS-ON LAB

Take the Lecture 6 ResNet, export it with its preprocessing config, build a complete FastAPI service (upload, batch, health, metrics endpoints), write pytest tests with `TestClient`, and run a latency benchmark reporting p50/p95/p99 before and after batching.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. Export a deployable artefact
2. The predict endpoint
3. Input validation
4. Batch endpoint
5. Test suite
6. Latency benchmark

READING

- FastAPI documentation: Request Files, Dependencies, Testing, Lifespan Events.
- PyTorch documentation: Saving and Loading Models; TorchScript.
- Sculley et al. (2015), *Hidden Technical Debt in Machine Learning Systems*.

data: shapes_32.npz, shapes_imagefolder/

slides: 6 sections

FastAPI service

LECTURE 10 · Part III - Vision Systems and Deployment

Object Detection*From 'what is in this image' to 'what, and exactly where'.***LEARNING OBJECTIVES**

- Define the detection task and the role of IoU, non-maximum suppression and mean average precision.
- Contrast two-stage (R-CNN family) with one-stage (YOLO, SSD, RetinaNet) detectors.
- Implement a single-scale anchor-free detector with classification and box-regression heads.
- Compute mAP correctly and interpret a precision-recall curve.

LECTURE OUTLINE

1. **The task and why it is harder** — Variable number of outputs per image - a classification head cannot express that.
2. **Intersection over Union** — $\text{IoU} = \text{area of overlap} / \text{area of union}$. The universal matching criterion.
3. **Two-stage detectors** — R-CNN: region proposals, then a CNN per region. Accurate and extremely slow.
4. **One-stage detectors** — YOLO: a single pass predicts boxes and classes on a grid. Fast enough for video.
5. **Non-maximum suppression** — Detectors emit many overlapping boxes for one object.
6. **Losses and evaluation** — Classification: cross-entropy or focal loss, over grid locations.

HANDS-ON LAB

Implement IoU and NMS from scratch with tests, build a single-scale anchor-free detector (a centre heatmap plus a size regression head) on the 96x96 multi-shape scenes, train it, decode its output, and implement mAP@0.5 to evaluate it properly.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. IoU, vectorised
2. Non-maximum suppression
3. Build the detector
4. Decode predictions
5. mean Average Precision
6. Confidence threshold sweep

READING

- Ren et al. (2015), *Faster R-CNN*.
- Redmon et al. (2016), *You Only Look Once*.
- Lin et al. (2017), *Focal Loss for Dense Object Detection*.
- Tian et al. (2019), *FCOS: Fully Convolutional One-Stage Object Detection*.

data: shapes_det.npz, shapes_det_annotations.json

slides: 6 sections

LECTURE 11 · Part III - Vision Systems and Deployment

Semantic Segmentation and Dense Prediction*One prediction per pixel, and the encoder-decoder that makes it possible.***LEARNING OBJECTIVES**

- Distinguish semantic, instance and panoptic segmentation and their evaluation metrics.
- Explain why an encoder-decoder with skip connections outperforms naive upsampling.
- Implement U-Net from scratch and train it on multi-object scenes.
- Compute mean IoU and Dice, and apply loss functions that handle class imbalance.

LECTURE OUTLINE

1. **Three segmentation tasks** — Semantic: one class label per pixel; two adjacent cars are one 'car' region.
2. **The resolution problem** — Classification backbones downsample aggressively - 32x is typical - to build semantic depth.
3. **Encoder-decoder with skip connections** — Encoder: downsample, growing the receptive field and the semantic content.
4. **Upsampling operators** — Nearest and bilinear interpolation: parameter-free, no checkerboard artefacts.
5. **Losses for dense prediction** — Pixel-wise cross-entropy is the default but is dominated by background pixels.
6. **Evaluation** — Per-class IoU = $TP / (TP + FP + FN)$, then mean IoU over classes.

HANDS-ON LAB

Implement U-Net from scratch, train it on the 96x96 shape scenes for 5-class segmentation, compare cross-entropy against Dice against the combination, implement mean IoU and Dice metrics, and show the effect of removing the skip connections.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. U-Net implementation
2. Train and visualise
3. mIoU and Dice from scratch
4. Loss comparison
5. Skip connections ablation
6. Class imbalance

READING

- Ronneberger et al. (2015), *U-Net: Convolutional Networks for Biomedical Image Segmentation*.
- Long et al. (2015), *Fully Convolutional Networks for Semantic Segmentation*.
- Chen et al. (2018), *DeepLabv3+*.
- Odena et al. (2016), *Deconvolution and Checkerboard Artifacts*.

data: shapes_seg.npz

slides: 6 sections

LECTURE 12 · Part III - Vision Systems and Deployment

Sequence Models: RNNs, LSTMs and Image Captioning*Memory, and the reason it eventually was not enough.***LEARNING OBJECTIVES**

- Explain recurrence, parameter sharing across time, and backpropagation through time.
- Derive why vanilla RNN gradients vanish, and how gating in an LSTM or GRU addresses it.
- Build a character-level language model and a CNN-encoder / RNN-decoder captioning model.
- Articulate the sequential bottleneck that motivated the Transformer.

LECTURE OUTLINE

1. **Recurrence** — $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b)$ - the same weights at every timestep.
2. **Backpropagation through time** — Unroll the network over the sequence and apply standard backpropagation.
3. **LSTM** — A cell state runs through the sequence with only additive interactions - an uninterrupted gradient path.
4. **Text as input** — Tokenisation: character, word, or subword (BPE / WordPiece). Subword is the modern default.
5. **Image captioning** — CNN encoder produces a feature vector; RNN decoder generates tokens conditioned on it.
6. **The bottleneck** — Recurrence is inherently sequential - timestep t cannot start before $t-1$ finishes. No parallelism.

HANDS-ON LAB

Implement an RNN cell and an LSTM cell from scratch and verify against PyTorch; train a character-level language model on the corpus and sample from it; demonstrate the vanishing-gradient difference between RNN and LSTM on a long-range copy task; then build CNN-encoder / LSTM-decoder captioning on the shapes images with greedy and beam-search decoding.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. RNN cell from scratch
2. LSTM cell from scratch
3. Character language model
4. Long-range copy task
5. Gradient flow comparison
6. Image captioning

READING

- Hochreiter & Schmidhuber (1997), *Long Short-Term Memory*.
- Karpathy (2015), *The Unreasonable Effectiveness of Recurrent Neural Networks*.
- Vinyals et al. (2015), *Show and Tell: A Neural Image Caption Generator*.
- Pascanu et al. (2013), *On the Difficulty of Training Recurrent Neural Networks*.

data: corpus.txt, sentiment.csv, captions.csv, shapes_32.npz

slides: 6 sections

Part IV - Attention, Generation and Production

LECTURE 13 · Part IV - Attention, Generation and Production

Attention and the Transformer*Scaled dot-product attention, built from nothing.***LEARNING OBJECTIVES**

- Derive scaled dot-product attention and explain every term, including the square-root scaling.
- Implement multi-head self-attention, positional encoding and a full encoder block from scratch.
- Distinguish encoder-only, decoder-only and encoder-decoder architectures and their uses.
- Analyse attention's quadratic cost and name the main mitigations.

LECTURE OUTLINE

1. **The idea** — Every position attends to every other position directly. Path length between any two tokens is 1.
2. **Multi-head attention** — Project into h subspaces, attend independently in each, concatenate, project out.
3. **Position** — Attention is permutation-equivariant - it has no notion of order at all.
4. **The Transformer block** — Sub-layer 1: multi-head self-attention. Sub-layer 2: position-wise feed-forward (usually 4x width).
5. **Three families** — Encoder-only (BERT): bidirectional context, for classification and retrieval.
6. **The cost** — Attention is $O(n^2)$ in sequence length, in both time and memory.

HANDS-ON LAB

Implement scaled dot-product attention, multi-head attention, sinusoidal positional encoding and a full pre-norm encoder block from scratch, verifying each against `torch.nn.MultiheadAttention`; train a Transformer classifier on the sentiment corpus; visualise attention weights; and measure the quadratic scaling empirically.

CORE TASKS (7 REQUIRED, 2 STRETCH)

1. Scaled dot-product attention
2. Why $\sqrt{d_k}$
3. Multi-head attention
4. Positional encoding
5. Encoder block and classifier
6. Attention visualisation
7. Causal masking

READING

- Vaswani et al. (2017), *Attention Is All You Need*.
- Alammari, *The Illustrated Transformer*.
- Xiong et al. (2020), *On Layer Normalization in the Transformer Architecture*.
- Dao et al. (2022), *FlashAttention*.

data: sentiment.csv, corpus.txt

slides: 6 sections

LECTURE 14 · Part IV - Attention, Generation and Production

Vision Transformers and Self-Supervised Learning*Images as sequences of patches, and learning without labels.***LEARNING OBJECTIVES**

- Explain the ViT patch-embedding pipeline and where its inductive bias differs from a CNN's.
- Implement a Vision Transformer from scratch and train it on the shapes dataset.
- Describe contrastive (SimCLR, MoCo), distillation (DINO) and masked (MAE) self-supervision.
- Run a linear probe to quantify the quality of a learned representation.

LECTURE OUTLINE

1. **Vision Transformer** — Split the image into fixed patches (16x16 in the original), flatten, and linearly project each.
2. **Inductive bias and the data requirement** — A CNN has locality and translation equivariance baked in. ViT has almost none.
3. **Why self-supervision** — Labels are expensive; unlabelled images are effectively free.
4. **Contrastive learning** — SimCLR: two augmented views of one image are positives; every other image in the batch is a negative.
5. **Beyond contrastive** — BYOL and DINO work without negatives, using a momentum teacher and stop-gradient to avoid collapse.
6. **Evaluating a representation** — Linear probe: freeze the encoder, train a linear classifier. The standard measure of representation quality.

HANDS-ON LAB

Implement patch embedding, the [CLS] token and a full ViT from scratch, train it on shapes and compare against the Lecture 6 CNN at matched parameter counts; then run SimCLR pre-training on the unlabelled pool and measure linear-probe accuracy against a supervised model trained on the same small label budget.

CORE TASKS (6 REQUIRED, 2 STRETCH)

1. Patch embedding
2. Vision Transformer
3. ViT versus CNN under data scarcity
4. Attention distance
5. SimCLR augmentations and NT-Xent
6. Pre-train and probe

READING

- Dosovitskiy et al. (2021), *An Image Is Worth 16x16 Words* (ViT).
- Chen et al. (2020), *A Simple Framework for Contrastive Learning* (SimCLR).
- He et al. (2022), *Masked Autoencoders Are Scalable Vision Learners*.
- Caron et al. (2021), *Emerging Properties in Self-Supervised Vision Transformers* (DINO).

data: shapes_32.npz, shapes_pairs.npz

slides: 6 sections

LECTURE 15 · Part IV - Attention, Generation and Production

Generative Models: Autoencoders, VAEs, GANs and Diffusion*Learning the distribution, not just the decision boundary.***LEARNING OBJECTIVES**

- Contrast discriminative and generative modelling and the trade-offs between the major families.
- Derive the VAE evidence lower bound and implement the reparameterisation trick.
- Train a GAN, recognise mode collapse, and apply the standard stabilisation techniques.
- Explain the forward and reverse diffusion processes and implement a minimal DDPM.

LECTURE OUTLINE

1. **The generative families** — Autoregressive (PixelCNN, GPT): exact likelihood, slow sequential sampling.
2. **Autoencoders** — Encoder compresses to a bottleneck; decoder reconstructs. Trained on reconstruction error.
3. **Variational autoencoders** — Encode to a distribution - a mean and a variance - rather than a point.
4. **Generative adversarial networks** — Generator versus discriminator, a minimax game with a Nash equilibrium as the goal.
5. **Diffusion models** — Forward process: add Gaussian noise over T steps until the image is pure noise. Fixed, no learning.
6. **Choosing, and the ethics** — Need likelihoods? Autoregressive or flows. Need speed? VAE or GAN. Need quality? Diffusion.

HANDS-ON LAB

Train an autoencoder and visualise its latent space; implement a VAE with the reparameterisation trick and interpolate between latent codes; train a DCGAN on shapes and deliberately induce and then fix mode collapse; implement a minimal DDPM with a small U-Net and visualise the reverse process step by step.

CORE TASKS (7 REQUIRED, 2 STRETCH)

1. Autoencoder and latent space
2. VAE with reparameterisation
3. Sample and interpolate
4. beta-VAE
5. DCGAN
6. Mode collapse
7. Minimal DDPM

READING

- Kingma & Welling (2014), *Auto-Encoding Variational Bayes*.
- Goodfellow et al. (2014), *Generative Adversarial Networks*.
- Ho et al. (2020), *Denoising Diffusion Probabilistic Models*.
- Radford et al. (2016), *DCGAN*.
- Ho & Salimans (2022), *Classifier-Free Diffusion Guidance*.

data: shapes_32.npz, digits_8x8.npz

slides: 6 sections

LECTURE 16 · Part IV - Attention, Generation and Production

Multimodal Learning and Production ML Systems*Joining vision to language, and shipping the result responsibly.***LEARNING OBJECTIVES**

- Explain contrastive vision-language pre-training and how it enables zero-shot classification.
- Implement a small CLIP-style dual encoder and perform zero-shot and retrieval tasks.
- Design a production ML system: versioning, monitoring, drift detection and rollback.
- Audit a model for robustness, fairness and calibration before release.

LECTURE OUTLINE

1. **Multimodal learning** — Fusion strategies: early (concatenate inputs), late (combine predictions), joint embedding.
2. **What multimodal buys you** — Zero-shot transfer to classes never explicitly labelled during training.
3. **Production architecture** — Offline training pipeline versus the online serving path - different constraints, different code.
4. **Monitoring** — Operational: latency percentiles, throughput, error rate, saturation.
5. **Pre-release audit** — Robustness: corruption benchmarks, adversarial examples, out-of-distribution detection.
6. **Where the field is going** — Scaling laws, and the growing dominance of compute-optimal training.

HANDS-ON LAB

Build a CLIP-style dual encoder over the shapes images and their captions, train it with symmetric InfoNCE, run zero-shot classification from text prompts and bidirectional retrieval; then build the production layer: a versioned model registry, a FastAPI service with monitoring and drift detection endpoints, and a complete pre-release audit with a model card.

CORE TASKS (8 REQUIRED, 2 STRETCH)

1. Dual encoder
2. Symmetric contrastive loss
3. Zero-shot classification
4. Prompt sensitivity
5. Bidirectional retrieval
6. Model registry and serving
7. Drift detection
8. Model card

READING

- Radford et al. (2021), *Learning Transferable Visual Models From Natural Language Supervision* (CLIP).
- Mitchell et al. (2019), *Model Cards for Model Reporting*.
- Sculley et al. (2015), *Hidden Technical Debt in Machine Learning Systems*.
- Hendrycks & Dietterich (2019), *Benchmarking Neural Network Robustness to Common Corruptions*.

data: shapes_32.npz, captions.csv, shapes_pairs.npz

slides: 6 sections

FastAPI service

6. Appendix

Task and dataset index

#	Lecture	Core	Stretch	Datasets
01	Deep Learning Foundations and the Image Data Pipeline	4	2	shapes_32.npz, shapes_imagefolder/
02	From Linear Models to Neural Networks	5	2	digits_8x8.npz, shapes_32.npz
03	Backpropagation and Automatic Differentiation	5	2	digits_8x8.npz, shapes_32.npz
04	Training Deep Networks: Optimisation and Regularisation	5	2	shapes_32.npz
05	Convolutional Neural Networks	6	2	shapes_32.npz
06	Modern CNN Architectures and Transfer Learning	5	2	shapes_32.npz, shapes_imagefolder/
07	Data Augmentation, Training Recipes and Experiment Tracking	6	2	shapes_32.npz
08	Evaluation, Interpretability and Model Debugging	6	2	shapes_32.npz, shapes_det.npz
09	Serving Deep Learning Models with FastAPI	6	2	shapes_32.npz, shapes_imagefolder/
10	Object Detection	6	2	shapes_det.npz, shapes_det_annotations.json
11	Semantic Segmentation and Dense Prediction	6	2	shapes_seg.npz
12	Sequence Models: RNNs, LSTMs and Image Captioning	6	2	corpus.txt, sentiment.csv, captions.csv, shapes_32.npz
13	Attention and the Transformer	7	2	sentiment.csv, corpus.txt
14	Vision Transformers and Self-Supervised Learning	6	2	shapes_32.npz, shapes_pairs.npz
15	Generative Models: Autoencoders, VAEs, GANs and Diffusion	7	2	shapes_32.npz, digits_8x8.npz
16	Multimodal Learning and Production ML Systems	8	2	shapes_32.npz, captions.csv, shapes_pairs.npz
Total		94	32	

Instructor notes

- Each notebook maps section-for-section onto its slide deck, so you can present the deck and run the matching notebook cells live.
- Cells taking more than about a minute on a laptop are marked in the notebook. Reduce the epoch count in class and let students run the full version later.
- Every result quoted in the slides was produced by running the notebook on CPU. If a number differs on your machine, the seed and thread count are the first things to check.
- Lectures 9 and 16 need no extra setup beyond `train_and_export.py`; the FastAPI service runs in-process via `TestClient` inside the notebook, so no port needs to be free during class.
- The running `results.md` table students maintain across all 16 lectures is the single most useful artefact they produce. Insist on it from Lecture 1.

Sources consulted for curriculum design

- Stanford CS231n, *Deep Learning for Computer Vision* — lecture schedule and assignment structure.
- NYU DS-GA 1008, *Deep Learning* (Yann LeCun, Alfredo Canziani) — theme organisation and the energy-based perspective.
- deeplearning.ai *Deep Learning Specialization* (Andrew Ng) — the foundations sequence and practical training recipes.
- fast.ai *Practical Deep Learning for Coders* — the top-down teaching approach and transfer-learning emphasis.
- Goodfellow, Bengio and Courville, *Deep Learning* (MIT Press) — the reference text throughout.